

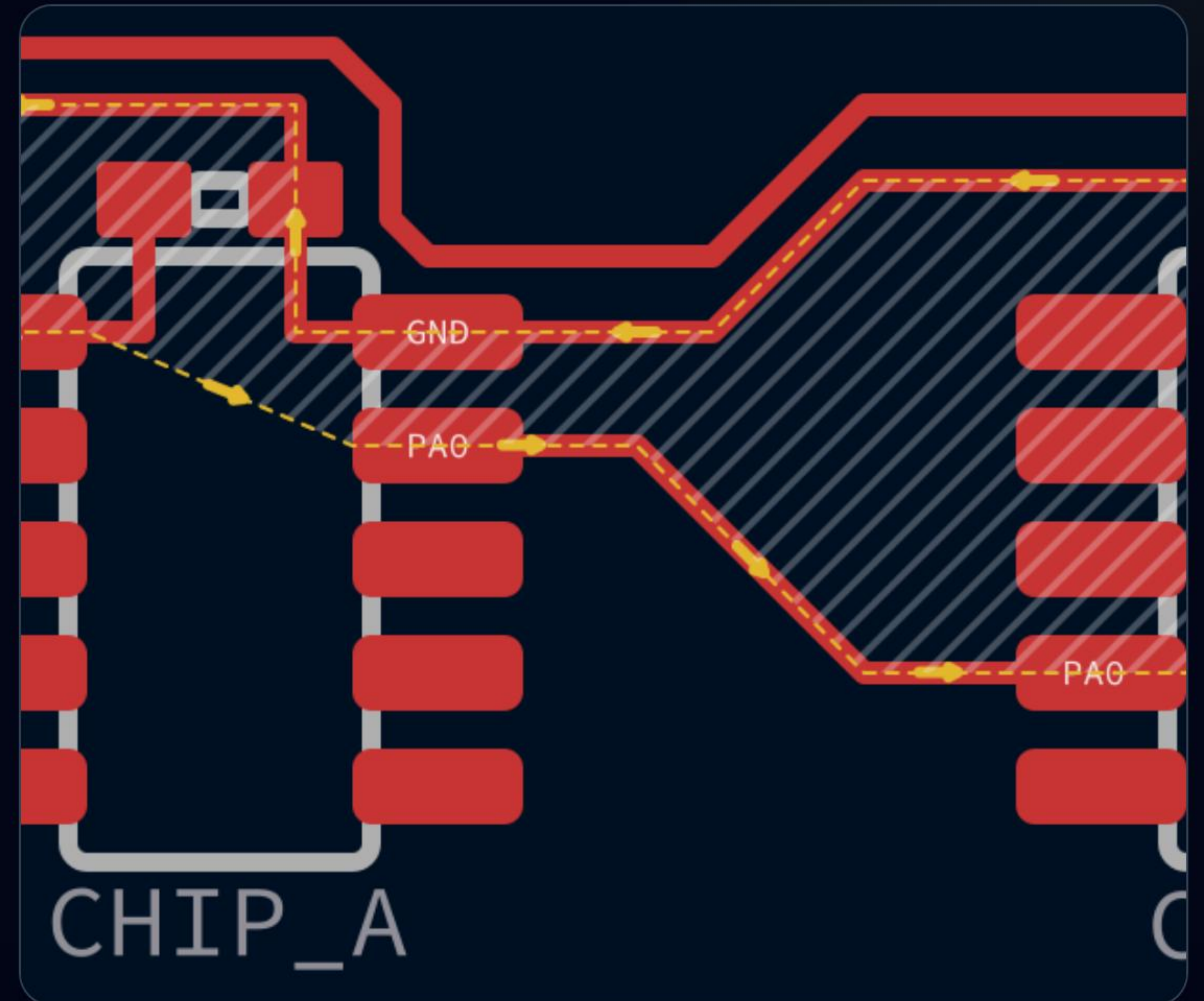
| WHY DO WE NEED CONDITIONALS?

The Problem of Choice

A program for a vending machine must decide: if money is inserted, dispense; otherwise, show "Insufficient funds".

Without conditionals, programs follow the same steps blindly, unable to adapt to different situations.

Benefit	Explanation
Decision-making	Respond to varied inputs/states.
Flexibility	One code handles many scenarios.
User Interaction	Reacts to choices or errors.



| THE "IF" STATEMENT



The Simplest Form

Checks a condition. If **True**, it runs the block. If **False**, it skips it.

Requirement: The colon **:** follows the condition, and the code block *must* be indented.

```
# Syntax structure
if condition:
    # code to run if True

# Practical Example
temperature = 18
if temperature < 20:
    print("It's cold outside!")
```

Mental Model: A gate. If the condition passes, you go through; otherwise, you bypass it.

| THE "IF-ELSE" STATEMENT

Defining Two Paths

When you want to do one thing for success and another for failure, use `else`.

- ✔ **Mutually Exclusive:** You take exactly one path.
- ▼ **Catch-all:** The `else` block handles everything that failed the `if`.

```
# Example usage
age = 16
if age ≥ 18:
    print("You can vote.")
else:
    print("You are too young to vote.")
```

Thinking of it as a fork in the road where you must choose one direction.

| THE IF-ELIF-ELSE CHAIN

Multiple Conditions

Used when you have more than two possible outcomes. Short for "else if".

- ↓ **Top-Down:** Checked sequentially.
- 👉 **Early Exit:** First **True** block wins; the rest are ignored.
- ⊕ **Scale:** Use as many **elif** as needed.

```
score = 85
if score ≥ 90:
    grade = "A"
elif score ≥ 80:
    grade = "B"
elif score ≥ 70:
    grade = "C"
else:
    grade = "F"
print(f"Grade: {grade}") # Output: B
```

| COMPARISON & LOGIC

Operator	Meaning
<code>=</code> / <code>≠</code>	Equal / Not Equal
<code>></code> / <code><</code>	Greater / Less
<code>≥</code> / <code>≤</code>	Greater-Equal / Less-Equal
<code>and</code> / <code>or</code>	Logic Gates (Both / Either)

Operator Precedence

1. `not` (highest)
2. `and`
3. `or` (lowest)

```
# Combining conditions
if age ≥ 18 and has_license:
    print("Drive authorized.")
```

| NESTED CONDITIONALS

Decisions Inside Decisions

Put an `if` inside another to handle layered logic.

 **Access:** Inner check only runs if outer check is `True`.

 **Limit:** Avoid nesting deeper than 2-3 levels for readability.

```
if age ≥ 18:
    if has_id:
        print("Welcome to the club!")
    else:
        print("You need an ID.")
else:
    print("You are too young.")
```

| PROFESSIONAL PATTERNS



Guard Clause

Check for errors/nulls early and `return` to keep logic flat.



Ternary Op

```
status = "adult" if age ≥ 18  
else "minor"
```



Chaining

Python allows `if 5 < x < 10:` instead of complex logic.

Common Mistake

Using `=` instead of `==`

Indentation Errors

Explanation

`=` is for assignment; will cause errors in conditions.

Python relies on spaces to define block scope.

| HANDS-ON PREVIEW

Problem: Value Analysis

Check if a number is positive, negative, or zero. If positive, also check parity.

- 1 **Analyze:** Define thresholds (0).
- 2 **Structure:** Use if-elif-else.
- 3 **Code:** Implement and indent.
- 4 **Test:** Input 5, -3, and 0.

```
number = int(input("Enter number: "))

if number > 0:
    print("Positive")
    if number % 2 == 0:
        print("and even")
    else:
        print("and odd")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

| IMAGE SOURCES



Thumbnail https://miro.medium.com/1*3VDHEUaYzV4LJBdT7KZHrA.png

for Source: medium.com

medium.com